

Code Explanation

Code Explanation: AWS Glue PySpark Job Test Suite
This is a comprehensive test suite for an AWS Glue PySpark ETL job. It uses mocking to test all transformations without requiring actual AWS resources or Spark I/O operations.---

Setup and Initialization

****Lines 1-27: Imports and Spark Session Setup****- Imports testing libraries (`unittest.mock``), PySpark components, and system modules- Adds the source directory to Python path for importing the job module- Creates a local Spark session with 2 cores for running tests in isolation---

Individual Component Tests

1. ****test_get_job_parameters()**** (Lines 30-39)

- ****Purpose****: Validates that job parameters are loaded correctly- ****Verification****: Checks that required parameter keys exist (customer path, order path, database name)- ****Confirms****: Default database name is ``gen_ai_poc_databrickscoe``

2. ****test_validate_s3_path()**** (Lines 42-51)

- ****Purpose****: Tests S3 path validation logic- ****Tests****: Valid S3 paths (``s3://``, ``s3a://``), invalid local paths, empty strings, and None values- ****Expected****: Returns True only for valid S3 URIs

3. ****test_normalize_columns()**** (Lines 54-73)

- ****Purpose****: Verifies column name normalization (converts to lowercase)- ****Process****: Creates DataFrame with mixed-case columns (``CustId``, ``Name``, ``EmailId``, ``Region``)- ****Verification****: Confirms all columns are converted to lowercase and original names are removed

4. ****test_apply_data_quality()**** (Lines 76-110)

- ****Purpose****: Tests data quality rules application- ****Test Data****: Includes nulls, "Null" strings, and duplicate records- ****Quality Rules****: Remove nulls, remove "Null" strings, remove duplicates- ****Expected Result****: Only 2 valid unique records remain after cleaning

5. ****test_ingest_customer_data()**** (Lines 113-140)

- ****Purpose****: Tests customer data ingestion with mocked S3 read- ****Mock Strategy****: Patches ``DataReader.read_csv`` to return test DataFrame instead of reading from S3- ****Verification****: Confirms 3 records loaded and columns are normalized to lowercase

6. ****test_ingest_order_data()**** (Lines 143-173)

- ****Purpose****: Tests order data ingestion with mocked S3 read- ****Mock Strategy****: Similar to customer test, mocks CSV read operation- ****Verification****: Confirms 3 order records loaded with correct normalized column names---

Data Processing Tests

7. **test_clean_data()** (Lines 176-220)

- **Purpose**: Tests the combined data cleaning function for both datasets- **Test Data**: Customer data with "Null" string, order data with valid records- **Process**: Applies all data quality rules to both DataFrames- **Expected**: Customer count reduced to 2 (removed "Null" string), orders remain at 2

8. **test_register_catalog_tables()** (Lines 223-263)

- **Purpose**: Tests Glue Catalog table registration with mocked writes- **Mock Strategy**: Patches `DataWriter._write_parquet` to avoid actual S3 writes- **Verification**: Confirms write method called twice (once for customer, once for order table)

9. **test_implement_scd_type2()** (Lines 266-319)

- **Purpose**: Tests Slowly Changing Dimension Type 2 implementation using Hudi- **SCD Columns**: Adds `IsActive`, `StartDate`, `EndDate`, `OpTs` for tracking changes- **Mock Strategy**: Patches `DataWriter._write_hudi` to avoid actual Hudi write- **Verification**: Confirms Hudi write called once for order summary table

10. **test_calculate_customer_aggregate_spend()** (Lines 322-368)

- **Purpose**: Tests customer spending aggregation calculation- **Test Data**: Multiple orders for customer 1, single order for customer 2- **Process**: Joins orders with customers and calculates total spend per customer- **Mock Strategy**: Patches parquet write to avoid S3 output- **Verification**: Confirms aggregation write called once---

Helper Class Tests

11. **test_data_reader_class()** (Lines 371-392)

- **Purpose**: Tests the DataReader utility class- **Mock Strategy**: Patches `spark.read.csv` method- **Verification**: Confirms CSV read returns expected DataFrame structure

12. **test_data_writer_class()** (Lines 395-425)

- **Purpose**: Tests the DataWriter utility class for multiple formats- **Tests Three Formats**: - Parquet write - CSV write - Hudi write with options- **Verification**: Confirms appropriate write method called for each format type---

End-to-End Integration Test

13. **test_end_to_end_workflow()** (Lines 428-542)

- **Purpose**: Tests complete ETL pipeline from ingestion to output- **Workflow Steps**: 1. Ingest customer data (2 records) 2. Ingest order data (3 records) 3. Clean both datasets 4. Register catalog tables 5. Implement SCD Type 2 for order summary 6. Calculate customer aggregate spend- **Mock Strategy**: Patches all I/O operations (reads and writes)- **Verification**: - Confirms correct record counts at each stage - Verifies parquet writes called at least 2 times (catalog + analytics) - Verifies Hudi

write called once (SCD Type 2)---

Test Execution

****Lines 545-577: Main Execution Block****- Runs all 13 tests sequentially when script executed directly- Prints success message for each passing test- Displays final summary with visual separator---

Key Testing Patterns Used

1. ****Mocking****: All AWS S3 and Glue Catalog operations are mocked to enable local testing2. ****Isolation****: Each test is independent with its own test data3. ****Verification****: Tests check both data correctness and method call counts4. ****Coverage****: Tests span data ingestion, transformation, quality, and output stagesThis test suite ensures the Glue job works correctly without requiring AWS infrastructure or incurring costs.